Национальный исследовательский ядерный университет «МИФИ»
Институт интеллектуальных кибернетических систем

ОТЧЁТ

по сессионному заданию
по дисциплине «Основы криптографии»

Ответы на теоретические вопросы и решение задач

Студент группы М26-545
Виноградов А. В.
Номер в списке группы: 27

Москва 2026

Содержание

Вопрос 1. Трёхсторонний протокол распределения ключей 3
Вопрос 2. Формальные модели шифров 7
Вопрос 3. Принципы построения поточных систем шифрования 12
Вопрос 4. Режим, в котором биективность шифрующих отображений не нужна 18
Вопрос 5. Распространение ошибки в режиме CFB с 8-битовыми блоками .. 20
Вопрос 6. Длина значения хэш-функции и вероятность коллизии 23

Вопрос 1. Трёхсторонний протокол распределения ключей

Задание: опишите трёхсторонний протокол распределения ключей между парами участников с использованием третьей стороны.

Постановка задачи. В сети из n участников любая пара должна иметь возможность получить общий секретный ключ для симметричного шифрования. Если раздать ключи всем парам заранее, понадобится $n(n-1)/2$ ключей, каждый участник будет хранить $n-1$ ключ, а появление нового участника потребует раздать новые ключи всем остальным. Трёхсторонний протокол решает задачу иначе: каждый участник заранее получает только один долговременный ключ, общий с доверенной третьей стороной, а ключ для конкретной пары третья сторона вырабатывает по запросу в момент установления связи. Всего долговременных ключей n — по одному на участника.

Участники и их роли.

- Алиса (A) — инициатор: хочет установить защищённую связь с Бобом.
- Боб (B) — второй участник пары, с которым Алиса хочет связаться.
- KDC — доверенная третья сторона, центр распределения ключей (англ. Key Distribution Center). KDC хранит секретные ключи всех зарегистрированных участников и вырабатывает сеансовые ключи. Алиса и Боб доверяют центру: считается, что он выбирает ключи случайно, никому их не раскрывает и не выдаёт себя за участников.

Исходные условия. До начала протокола у Алисы и KDC есть общий секретный ключ K_A, у Боба и KDC — ключ K_B. Эти долговременные ключи доставлены по защищённому каналу при регистрации участников. Общего ключа у Алисы и Боба нет. Запись $E_K(M)$ означает сообщение M, зашифрованное симметричным шифром на ключе K. Шифрование должно также обнаруживать изменение шифртекста (например, вместе с

имитовставкой), иначе противник сможет незаметно менять поля сообщений. R_A и R_B — одноразовые случайные числа (nonce) Алисы и Боба.

Базовая схема. Общий порядок работы центра такой: Алиса запрашивает KDC, центр сообщает Бобу о запросе Алисы, и при согласии Боба создаётся ключ сеанса. В простейшем протоколе создания сеансового ключа сообщение центра для Боба доставляет сама Алиса, и протокол состоит из трёх шагов (таблица 1).

Таблица 1 – Протокол создания ключа сеанса с помощью KDC

Шаг	**Кто → кому**	**Сообщение**	**Смысл шага**
1	A → KDC	A, B	Алиса открыто сообщает центру, что хочет связаться с Бобом
2	KDC → A	$E_{K_A}\left(K_{AB}, E_{K_B}(A, B, K_{AB})\right)$	KDC вырабатывает случайный сеансовый ключ K_{AB} и отправляет его Алисе вместе с «билетом» для Боба
3	A → B	$E_{K_B}(A, B, K_{AB})$	Алиса пересылает билет Бобу, Боб расшифровывает его ключом K_B и узнаёт K_{AB}

После шага 3 ключ K_{AB} есть у обоих, но у схемы три недостатка. Во-первых, в сообщениях нет ничего, что подтверждало бы их свежесть: противник, который записал сообщение шага 2 из старого сеанса и узнал старый сеансовый ключ, может выдать это сообщение Алисе за новый ответ центра, и Алиса начнёт шифровать на скомпрометированном ключе. Во-вторых, во внешней части ответа центра нет имени Боба, поэтому Алиса не может проверить, для связи с кем выдан ключ: противник, заменивший в открытом запросе шага 1 имя Боба своим, получит билет на свой ключ и сеансовый ключ для связи с Алисой. В-третьих, ни Алиса, ни Боб не убеждаются, что другая сторона действительно получила ключ. Протокол Нидхема — Шрёдера устраняет первые два недостатка для Алисы и добавляет подтверждение ключа; свежесть ключа для Боба в нём по-прежнему не проверяется (см. ниже).

Протокол Нидхема — Шрёдера (1978 г.) добавляет к базовой схеме случайные числа участников и шаги подтверждения ключа (таблица 2).

Таблица 2 – Шаги протокола Нидхема — Шрёдера

Шаг	**Кто → кому**	**Сообщение**	**Смысл шага**
1	A → KDC	A, B, R_A	Алиса сообщает центру, кто она и с кем хочет связаться, и прикладывает новое случайное число
2	KDC → A	$E_{K_A}\left(R_A, B, K_{AB}, E_{K_B}(A, K_{AB})\right)$	KDC вырабатывает случайный сеансовый ключ K_{AB} и отправляет его Алисе вместе с билетом для Боба
3	A → B	$E_{K_B}(A, K_{AB})$	Алиса пересылает билет Бобу
4	B → A	$E_{K_{AB}}(R_B)$	Боб проверяет, что собеседник владеет ключом
5	A → B	$E_{K_{AB}}(R_B - 1)$	Алиса подтверждает, что владеет ключом

На шаге 2 Алиса расшифровывает ответ своим ключом K_A и проверяет две вещи. Совпадение R_A с числом, отправленным на шаге 1, означает, что ответ свежий, а не записанный противником ответ из прошлого сеанса; так закрывается слабость базовой схемы. Имя B в ответе означает, что противник не подменил адресата в открытом сообщении шага 1, иначе Алиса получила бы ключ для связи с самим противником. Билет $E_{K_B}(A, K_{AB})$ Алиса прочитать не может, потому что он зашифрован ключом Боба; она хранит его и пересылает без изменений.

На шаге 3 Боб расшифровывает билет ключом K_B и узнаёт сеансовый ключ K_{AB} и имя Алисы. Сформировать такое сообщение мог только тот, кто знает K_B, то есть KDC. Поэтому Боб уверен, что ключ выдал центр и что ключ предназначен для связи с Алисой.

Шаги 4 и 5 — взаимное подтверждение ключа. Боб шифрует на K_{AB} своё случайное число, а Алиса должна вернуть его, изменённое по известному правилу ($R_B - 1$). Изменение нужно, чтобы ответом не могло служить само сообщение шага 4, отправленное противником обратно. После шага 5 Боб знает, что Алиса действительно получила K_{AB} и участвует в протоколе сейчас.

Результат протокола. Алиса и Боб получают общий секретный сеансовый ключ K_{AB}. Ключ выработал KDC, поэтому его знают только Алиса, Боб и KDC. Ключ шифрует данные одного сеанса связи между Алисой и Бобом; для следующего сеанса или для другой пары участников центр выработает новый ключ. Долговременные ключи K_A и K_B шифруют только короткие служебные сообщения, поэтому противник получает мало материала для их анализа.

Слабость протокола и её устранение. Деннинг и Сакко (1981 г.) показали, что в билете шага 3 нет ничего, по чему Боб мог бы проверить свежесть ключа. Пусть противник узнал сеансовый ключ K'_{AB} одного из прошлых сеансов и записал соответствующее сообщение шага 3. Тогда он может послать это сообщение Бобу повторно, расшифровать запрос шага 4 известным ему ключом K'_{AB} и правильно ответить на шаге 5, и Боб будет считать, что говорит с Алисой. Слабость устраняют, добавляя в билет метку времени (протокол Деннинга — Сакко) или случайное число Боба (так сделано в исправленном варианте самих Нидхема и Шрёдера 1987 г.). Тот же приём использует протокол Отвея — Рииса: Алиса сначала обращается к Бобу, Боб — к KDC, центр возвращает Бобу две части, в каждой из которых сеансовый ключ зашифрован вместе со случайным числом получателя, и Боб пересылает Алисе её часть. На схеме с метками времени построена система Kerberos: в ней билет содержит время выдачи и срок действия, а функции центра разделены между сервером аутентификации и сервером выдачи билетов.

Несколько центров. В большой сети один KDC становится узким местом и единой точкой отказа, поэтому центры объединяют в плоское множество (у каждой группы участников свой KDC, и центры взаимодействуют друг с другом) или в иерархическое (местные, национальные и международный центры). Схема для пары участников при этом та же, только запрос проходит через цепочку центров, поэтому у самих

центров должны быть общие ключи друг с другом (в плоском множестве — между центрами, в иерархии — с вышестоящим центром).

Третья сторона может играть и другую роль — центра трансляции ключей. Тогда сеансовый ключ вырабатывает сама Алиса, а центр только перешифровывает его с ключа K_A на ключ K_B и передаёт Бобу (так устроен протокол «широкоротой лягушки»). Итог тот же: общий сеансовый ключ пары Алиса — Боб.

Вопрос 2. Формальные модели шифров

Задание: приведите формальные модели шифров.

Чтобы строго рассуждать о шифре — можно ли однозначно расшифровать криптограмму, сколько у шифра различных ключей, что противник узнаёт из перехвата, — шифр описывают математической моделью. Основу такого подхода заложил К. Шеннон в работе «Теория связи в секретных системах» (1949 г.). Основные модели — алгебраическая и вероятностная. Кроме них используют обобщение на шифры с неоднозначным зашифрованием и модель шифрования сообщений произвольной длины, в которой ключ разворачивается в ключевой поток.

2.1 Алгебраическая модель

Шифр в алгебраической модели — совокупность

$$\Sigma_A = (X, K, Y, E, D),$$

компоненты которой:

- X — конечное множество открытых текстов (сообщений), например слова длины l в некотором алфавите или двоичные блоки длины n;
- K — множество ключей;
- Y — множество шифрованных текстов (криптограмм);

- $E = \{E_k: X \to Y \mid k \in K\}$ — правила зашифрования: каждый ключ k задаёт отображение E_k, и $y = E_k(x)$ — криптограмма текста x на ключе k;
- $D = \{D_k: Y \to X \mid k \in K\}$ — правила расшифрования.

Компоненты должны удовлетворять двум условиям:

1. $D_k(E_k(x)) = x$ для всех $x \in X$ и $k \in K$ — законный получатель, зная ключ, однозначно восстанавливает текст. Из этого условия следует, что каждое E_k инъективно: разные тексты на одном ключе дают разные криптограммы.
2. $Y = \bigcup_{k \in K} E_k(X)$ — в Y нет «лишних» элементов, каждая криптограмма может появиться при шифровании.

Шифр называют эндоморфным, если $X = Y$. Тогда каждое E_k — инъективное отображение конечного множества в себя, то есть подстановка на X. Так устроен блочный шифр: для каждого ключа E_k — подстановка на множестве V_n двоичных блоков длины n.

Небольшой шифр удобно задавать таблицей зашифрования: строки соответствуют ключам, столбцы — открытым текстам, в клетке стоит $E_k(x)$; по условию 1 в каждой строке все значения различны. Ключи k и k′ эквивалентны, если $E_k = E_{k'}$: они шифруют одинаково, и число реально различных правил шифрования может быть меньше $|K|$.

Пример: шифр гаммирования над Z_m, у которого $X = Y = K = Z_m^l$, $E_k(x) = (x_1 + k_1, \ldots, x_l + k_l) \bmod m$ и $D_k(y) = (y_1 - k_1, \ldots, y_l - k_l) \bmod m$.

Назначение модели: описать устройство шифра без вероятностей, проверить корректность расшифрования, подсчитать число различных правил шифрования, найти эквивалентные ключи, выделить классы шифров (замены, перестановки, гаммирования). Для оценки стойкости этой модели мало: стойкость зависит от того, какие тексты и ключи вероятны.

2.2 Вероятностная модель

Вероятностная модель дополняет алгебраическую распределениями вероятностей:

$$\Sigma_B = \big(X, K, Y, E, D, P(X), P(K)\big),$$

где:

- (X, K, Y, E, D) — шифр в алгебраической модели;
- P(X) — распределение вероятностей на открытых текстах; его даёт модель открытого текста, например позначная или марковская модель языка;
- P(K) — распределение вероятностей на ключах, обычно равномерное: ключ выбирается случайно;
- выбор текста и выбор ключа считаются независимыми.

Распределение на криптограммах получается из двух заданных:

$$P(y) = \sum_{k \in K} P\,(k) \sum_{x:\, E_k(x) = y} P\,(x).$$

Противник, перехвативший криптограмму y, может по формуле Байеса пересчитать вероятности открытых текстов:

$$P(x \mid y) = \frac{P(x) \sum_{k:\, E_k(x) = y} P\,(k)}{P(y)}.$$

Назначение модели — формально описать, что знает противник до перехвата (априорные вероятности P(x)) и после него (апостериорные P(x | y)), и на этой основе определить стойкость. По Шеннону шифр совершенно стоек, если P(x | y) = P(x) для всех x ∈ X и всех y ∈ Y с P(y) > 0: перехват не добавляет противнику никаких сведений о тексте. Шеннон доказал, что для совершенной стойкости ключей должно быть не меньше, чем возможных криптограмм, а криптограмм — не меньше, чем открытых текстов: |K| ≥ |Y| ≥ |X|. Пример совершенно стойкого шифра — шифр Вернама: X = Y = K = V_l,

$E_k(x) = x \oplus k$, ключ равновероятен и используется один раз. В этой же модели определяются энтропия ключа, ненадёжность ключа $H(K \mid Y)$ и расстояние единственности — длина криптограммы, начиная с которой ключ, как правило, определяется однозначно. В ней же оценивается имитостойкость — вероятности успешной имитации и подмены сообщения.

2.3 Шифры с неоднозначным зашифрованием

В алгебраической модели E_k — функция: текст на данном ключе всегда даёт одну и ту же криптограмму. В обобщённой модели $E_k(x) \subseteq Y$ — множество допустимых криптограмм, и при зашифровании из него выбирается одна, обычно случайно. Однозначность расшифрования сохраняется при условии

$$E_k(x) \cap E_k(x') = \varnothing \quad \text{при } x \neq x',$$

то есть множества криптограмм разных текстов на одном ключе не пересекаются, и по криптограмме получатель однозначно находит текст. Пример — омофонная замена: частая буква заменяется любым из нескольких символов, чтобы частоты символов шифртекста выровнялись. Назначение модели — описать рандомизированные шифры, в которых одинаковые тексты дают разные криптограммы и статистика открытого текста сглаживается.

2.4 Модель шифрования сообщений произвольной длины

Реальные сообщения — длинные последовательности знаков, а ключ один. Для них используют составную модель:

- A и B — алфавиты знаков открытого и шифрованного текста; знак — буква, бит, байт или двоичный блок длины n;
- $\Sigma = (A, Q, B, E, D)$ — опорный шифр, который шифрует один знак; $q \in Q$ — его ключ;

- K — множество ключей шифрсистемы; IV — открытый вектор инициализации (синхропосылка), если он есть;
- ψ — ключевой генератор: $\psi(k, IV) = q_1\ q_2\ q_3\ \ldots$ — ключевой поток, $q_i \in Q$;
- зашифрование текста $x = x_1\ \ldots\ x_l$: $y_i = E_{q_i}(x_i)$; расшифрование: $x_i = D_{q_i}(y_i)$.

Генератор удобно описывать как конечный автомат: множество состояний S, начальное состояние s_0, зависящее от k и IV, функция переходов h и функция выхода f: $q_i = f(s_i)$, $s_{i+1} = h(s_i)$. Если функция переходов зависит ещё и от знаков шифртекста, система самосинхронизирующаяся. Так как множество S конечно, ключевой поток синхронного генератора начиная с некоторого такта периодичен, а если функция переходов h биективна, — чисто периодичен.

Частные случаи:

- $q_i = k$ при всех i — шифр простой замены, например блочный шифр в режиме простой замены, где все блоки шифруются одним ключом;
- периодическая последовательность q_i — многоалфавитный шифр с периодическим ключом (шифр Виженера);
- опорный шифр — сложение по модулю 2, а ψ — генератор псевдослучайной гаммы — поточный шифр гаммирования.

Назначение модели — разделить простое преобразование знака и механизм выработки ключевого потока. Стойкость такой системы определяется свойствами ψ: периодом, статистикой и сложностью ключевого потока. На этой модели формулируются принципы построения поточных шифров (вопрос 3).

Сводка моделей приведена в таблице 3.

Таблица 3 – Формальные модели шифров

Модель	**Компоненты**	**Назначение**
Алгебраическая	X, K, Y, E, D; условия $D_k(E_k(x)) = x$ и Y — объединение $E_k(X)$	Устройство шифра, корректность расшифрования, эквивалентные ключи, классы шифров
Вероятностная	Алгебраическая модель, P(X), P(K), независимость текста и ключа	Знания противника до и после перехвата, совершенная стойкость, расстояние единственности, имитостойкость
С неоднозначным зашифрованием	$E_k(x)$ — множество криптограмм; множества разных текстов не пересекаются	Рандомизированные шифры, сглаживание статистики текста
Для сообщений произвольной длины	Опорный шифр, ключ, IV, ключевой генератор ψ (конечный автомат)	Описание реальных блочных и поточных шифрсистем, анализ ключевого потока

Вопрос 3. Принципы построения поточных систем шифрования

Задание: приведите принципы построения поточных систем шифрования.

Поточный шифр зашифровывает сообщение по одному знаку (биту или байту), причём каждый знак — своим элементом ключевого потока, который меняется от позиции к позиции. Поэтому одинаковые знаки открытого текста на разных позициях зашифровываются по-разному, в отличие от блочного шифра в режиме простой замены, где одинаковые блоки дают одинаковые блоки шифртекста. В модели из п. 2.4 поточный шифр — это опорный шифр для одного знака и ключевой генератор.

Почти все поточные системы построены на гаммировании:

$$y_i = x_i \oplus \gamma_i, \qquad x_i = y_i \oplus \gamma_i,$$

где γ_1, γ_2, … — гамма шифра. Если гамма — истинно случайная равновероятная последовательность длиной не меньше сообщения и используется один раз, получается шифр Вернама (одноразовый блокнот), который по теореме Шеннона совершенно стоек. Его недостаток —

секретный ключ такой же длины, как все передаваемые сообщения, и этот ключ надо заранее доставить получателю.

Отсюда главный принцип практических поточных систем: истинно случайная гамма заменяется псевдослучайной, которую детерминированный генератор вырабатывает из короткого секретного ключа k и открытого вектора инициализации IV: $\gamma = G(k, IV)$. Получатель с тем же ключом и тем же IV вырабатывает ту же гамму. Совершенная стойкость при этом теряется, потому что ключей меньше, чем сообщений, и стойкость системы целиком определяется генератором. Противник, знающий часть открытого текста, сразу получает соответствующий отрезок гаммы $\gamma_i = x_i \oplus y_i$. Поэтому генератор строят так, чтобы по известному отрезку гаммы нельзя было ни вычислить ключ, ни предсказать остальную гамму. Остальные принципы конкретизируют это требование и добавляют требования надёжности и эффективности.

3.1 Структурные принципы

1. Разделение генератора гаммы и операции наложения. Гамму накладывают простой обратимой операцией — сложением по модулю 2 или по модулю размера алфавита. Зачем: зашифрование и расшифрование выполняет одно и то же устройство, операция быстрая, а вся криптографическая сложность сосредоточена в генераторе, который можно анализировать отдельно.

2. Выбор способа синхронизации. Генераторы отправителя и получателя должны работать согласованно. Различают два класса систем.

- Синхронные: гамма зависит только от ключа и IV и не зависит от текста. Ошибка в бите шифртекста искажает только один бит открытого текста, то есть ошибки не размножаются; это важно для каналов с помехами (радиосвязь, сотовая связь). Гамму можно выработать заранее. Но если в канале потерян или вставлен знак,

генераторы расходятся, и весь дальнейший текст расшифровывается неверно, поэтому нужна внешняя синхронизация, например новая синхропосылка для каждого кадра.

- Самосинхронизирующиеся: знак гаммы зависит от ключа и d предыдущих знаков шифртекста, $\gamma_i = F(k, y_{i-d}, \ldots, y_{i-1})$. После потери или искажения знака синхронизация восстанавливается сама через d знаков, но одна ошибка искажает до d + 1 знаков открытого текста. Пример — режим CFB блочного шифра (вопрос 5).

Зачем: выбор между устойчивостью к сбоям синхронизации и отсутствием размножения ошибок делается по свойствам канала связи.

3. Однократное использование гаммы. Один и тот же отрезок гаммы нельзя использовать для двух сообщений. Если $y = x \oplus \gamma$ и $y' = x' \oplus \gamma$, то $y \oplus y' = x \oplus x'$: гамма исчезает, и оба текста восстанавливаются по избыточности языка. Поэтому ключ сочетают с вектором инициализации: ключ долговременный и секретный, а IV новый для каждого сообщения и передаётся открыто. Зачем: стойкость при многократном использовании одного ключа.

3.2 Требования к генератору гаммы

Эти требования составляют системно-теоретический подход к построению генераторов (Р. Рюппель): стойкость не доказывается математически, но генератор проверяют на устойчивость ко всем известным методам анализа.

4. Большой период. Генератор — конечный автомат, поэтому его выход рано или поздно становится периодическим. Повтор гаммы внутри сообщения или между сообщениями на одном ключе равносилен повторному использованию гаммы (принцип 3). Период должен быть гарантированно больше суммарной длины текстов, шифруемых на одном ключе. Например,

линейный регистр сдвига с обратной связью (LFSR) длины L с примитивным характеристическим многочленом даёт максимальный период $2^L - 1$.

5. Большая линейная сложность. Линейная сложность последовательности — длина самого короткого LFSR, который её порождает. По 2L знакам последовательности линейной сложности L алгоритм Берлекэмпа — Месси строит такой регистр, после чего противник вычисляет всю дальнейшую гамму. Поэтому LFSR сам по себе непригоден как генератор гаммы: его линейная сложность равна длине регистра. Линейная сложность гаммы должна быть огромной, а линейная сложность её начальных отрезков должна расти примерно как половина длины отрезка, как у случайной последовательности.

6. Хорошие статистические свойства. Гамма должна быть статистически неотличима от случайной равновероятной последовательности: одинаковые частоты нулей и единиц, серий и k-грамм, малая автокорреляция (постулаты Голомба, наборы тестов вроде NIST SP 800-22). Зачем: любой перекос в гамме переходит в шифртекст и даёт противнику статистическую атаку.

7. Нелинейность и корреляционная иммунность. Большую линейную сложность получают нелинейными узлами: комбинирующими генераторами (выходы нескольких LFSR проходят через нелинейную булеву функцию), фильтрующими генераторами (нелинейная функция от нескольких ячеек одного регистра), неравномерным движением регистров (сжимающий генератор, генератор «стоп-пошёл», A5/1 с мажоритарным управлением) или регистрами с нелинейной обратной связью (Trivium, Grain). Комбинирующая функция должна быть корреляционно-иммунной: её выход не должен коррелировать с выходом отдельного регистра или небольшой группы регистров. Иначе корреляционная атака Зигенталера восстанавливает начальные состояния регистров по отдельности: перебирает 2^L вариантов для каждого регистра вместо перебора всего ключа. При этом

есть компромисс: у булевой функции от n переменных с корреляционной иммунностью порядка m алгебраическая степень не превышает n − m (неравенство Зигенталера), так что повышение иммунности ограничивает степень функции, а с ней и линейную сложность гаммы.

8. Перемешивание и рассеивание. Каждый знак гаммы должен сложным образом зависеть от всех битов ключа и IV, а изменение одного бита ключа или IV должно менять в среднем половину битов гаммы. Для этого перед выработкой гаммы генератор проходит фазу инициализации: загружает ключ и IV и делает много холостых тактов, выход которых не используется (у Trivium — 1152 такта). Зачем: защита от атак на близкие ключи и векторы инициализации и от алгебраических атак.

9. Достаточный размер ключа и внутреннего состояния. Ключ должен быть достаточно длинным против полного перебора (сейчас не короче 128 бит), а внутреннее состояние генератора — не меньше удвоенной длины ключа. Иначе работают атаки, которые обменивают время на память (Бэббидж, Голич).

3.3 Эксплуатационные принципы

10. Эффективность реализации. Генератор строят из простых операций — сдвигов, сложений по модулю 2, табличных замен, — которые быстро выполняются и в аппаратуре, и программно. Поточный шифр не ждёт накопления целого блока и даёт минимальную задержку, поэтому подходит для потоковых данных (голос, видео) и устройств с ограниченными ресурсами. Высокая скорость и простота — основная причина, по которой поточные шифры применяют наряду с блочными.

11. Отдельная защита целостности. Гаммирование не защищает от изменения текста: инверсия бита шифртекста инвертирует тот же бит открытого текста, и противник, знающий формат сообщения, может предсказуемо его менять. Поэтому поточную систему дополняют

имитовставкой (кодом аутентификации сообщений) или используют схему аутентифицированного шифрования, например ChaCha20-Poly1305.

3.4 Подходы к обоснованию стойкости

Р. Рюппель выделяет четыре подхода к построению поточных шифров:

- системно-теоретический — описан выше: генератор строится по набору критериев и проверяется против известных атак; так построено большинство практических шифров;
- сложностно-теоретический — стойкость генератора сводится к трудной математической задаче; например, стойкость генератора Блюм — Блюма — Шуба доказывается в предположении, что разложение модуля на множители — трудная задача;
- информационно-теоретический — противник не получает сведений о тексте даже при неограниченных вычислительных ресурсах (шифр Вернама);
- рандомизированный — ключ указывает, какие части большой открытой случайной последовательности используются, и противнику приходится обрабатывать огромный объём данных.

Назначение каждого принципа сведено в таблицу 4.

Таблица 4 – Принципы построения поточных шифров и их назначение

№	Принцип	Что обеспечивает
1	Генератор гаммы и простое наложение	Эффективность, единое устройство для зашифрования и расшифрования
2	Выбор способа синхронизации	Надёжность связи: нет размножения ошибок или самовосстановление синхронизации
3	Однократное использование гаммы, IV	Стойкость при многократном использовании ключа
4	Большой период	Стойкость: нет повторов гаммы
5	Большая линейная сложность	Стойкость к восстановлению гаммы алгоритмом Берлекэмпа — Месси
6	Хорошая статистика	Стойкость к статистическим атакам
7	Нелинейность, корреляционная иммунность	Стойкость к линейным и корреляционным атакам
8	Перемешивание и рассеивание	Стойкость к атакам на связанные ключи и IV, алгебраическим атакам
9	Размер ключа и состояния	Стойкость к перебору и атакам «время — память»
10	Простые операции	Эффективность: скорость, малая задержка, малые ресурсы
11	Имитовставка	Защита целостности, которой гаммирование не даёт

Вопрос 4. Режим, в котором биективность шифрующих отображений не нужна

Задание: приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.

Почему в базовом режиме биективность нужна. Блочный шифр с длиной блока n задаёт для каждого ключа k отображение E_k: $V_n \to V_n$, где V_n — множество двоичных векторов длины n. В базовом режиме — режиме простой замены (ECB) — каждый блок шифруется отдельно, $y_i = E_k(x_i)$, а расшифрование выполняется обратным отображением: $x_i = E_k^{-1}(y_i)$. Если E_k не биекция, найдутся разные блоки $a \neq b$ с $E_k(a) = E_k(b)$, потому что для отображения конечного множества в себя неинъективность и несюръективность равносильны. Получатель, увидев блок $E_k(a)$, не сможет определить, был это a или b. Значит, в режиме простой замены (и в режиме

сцепления блоков CBC, где $x_i = E_k^{-1}(y_i) \oplus y_{i-1}$) биективность всех E_k необходима.

Пример — режим гаммирования с обратной связью по выходу (OFB). В этом режиме блочный шифр не шифрует открытый текст, а служит генератором гаммы. Отправитель выбирает вектор инициализации (синхропосылку) IV, который передаётся открыто, и вычисляет

$$\gamma_0 = IV, \qquad \gamma_i = E_k(\gamma_{i-1}), \qquad y_i = x_i \oplus \gamma_i, \quad i = 1,2,\dots,m.$$

Последний неполный блок текста складывается с началом γ_m.

Механизм расшифрования. Получатель знает ключ k и синхропосылку IV, поэтому вычисляет ту же последовательность γ_1, γ_2, …, применяя то же прямое отображение E_k, и восстанавливает текст: $x_i = y_i \oplus \gamma_i$.

Обоснование однозначности.

1. Гамма полностью определяется парой (k, IV): γ_i получается i-кратным применением E_k к IV. Для этого нужно только, чтобы E_k было отображением, то есть каждому аргументу соответствовало одно значение. Инъективность и сюръективность нигде не используются, обратное отображение E_k^{-1} не вычисляется ни при зашифровании, ни при расшифровании.
2. Открытый текст через E_k не проходит. Он участвует только в операции $x \mapsto x \oplus \gamma$, которая при любом фиксированном γ — биекция V_n, обратная сама себе: $(x \oplus \gamma) \oplus \gamma = x$. Следовательно, расшифрование возвращает исходный текст при любом x и при любом выборе функций E_k.

Даже в предельном случае постоянной функции $E_k(z) = c$ расшифрование остаётся однозначным ($\gamma_i = c$ при всех $i \geq 1$), хотя такой шифр, конечно, нестоек: все блоки шифруются одной и той же гаммой.

Где биективность всё же важна. Она влияет не на расшифрование, а на стойкость. Последовательность $\gamma_0, \gamma_1, \ldots$ — траектория отображения E_k на множестве из $N = 2^n$ элементов, и рано или поздно она зацикливается. Если E_k — подстановка, траектория чисто периодическая (возвращается к IV), и для случайной подстановки средняя длина цикла, содержащего IV, равна $(N + 1)/2$. Если E_k — случайное отображение, траектория имеет вид буквы ρ: сначала идёт «хвост», потом цикл, и первое повторение гаммы наступает в среднем примерно через $\sqrt{(\pi N/2)}$ блоков. При $n = 64$ это около $5{,}4 \cdot 10^9$ блоков вместо примерно $9{,}2 \cdot 10^{18}$. Повтор гаммы опасен (принцип 3 в вопросе 3), поэтому на практике в OFB используют блочный шифр, то есть подстановку, но для однозначного расшифрования это не требуется.

То же верно для режима гаммирования с обратной связью по шифртексту (CFB, вопрос 5) и для режима гаммирования со счётчиком (CTR): в них E_k тоже только вырабатывает гамму, а при расшифровании используется прямое отображение E_k. Поэтому в этих режимах вместо блочного шифра можно использовать любую псевдослучайную функцию, например построенную на хэш-функции.

Вопрос 5. Распространение ошибки в режиме CFB с 8-битовыми блоками

Задание: сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.

Исходные данные. Блочный шифр с длиной блока $n = 64$ бита работает в режиме гаммирования с обратной связью по шифртексту (CFB), в котором блоки (сегменты) открытого и шифрованного текста имеют длину $s = 8$ бит. В шифртексте искажён один бит.

Как работает режим. Используется сдвиговый регистр R длиной n = 64 бита; в начале в него записывается синхропосылка: $R_1 = IV$. На i-м шаге:

1. содержимое регистра шифруется блочным шифром: $O_i = E_k(R_i)$;
2. гамма — старшие 8 бит результата: $\gamma_i = MSB_8(O_i)$;
3. сегмент шифртекста: $y_i = x_i \oplus \gamma_i$;
4. регистр сдвигается на 8 бит влево, и справа в него записывается полученный сегмент шифртекста: $R_{i+1} = LSB_{56}(R_i) \parallel y_i$.

$$\gamma_i = \mathrm{MSB}_8\big(E_k(R_i)\big), \qquad y_i = x_i \oplus \gamma_i, \qquad R_{i+1} = \mathrm{LSB}_{56}(R_i) \parallel y_i.$$

Получатель держит такой же регистр, но заполняет его принятыми сегментами шифртекста: $x_i = y_i \oplus MSB_8(E_k(R_i))$, $R_{i+1} = LSB_{56}(R_i) \parallel y_i$.

Главное наблюдение: каждый сегмент шифртекста, попав в регистр, сдвигается на 8 бит за шаг и через 64/8 = 8 шагов выходит из него. Значит, начиная с девятого шага регистр содержит ровно восемь последних сегментов шифртекста:

$$R_i = y_{i-8} \parallel y_{i-7} \parallel \ldots \parallel y_{i-1},$$

и гамма γ_i зависит от ключа и восьми предыдущих сегментов шифртекста. Это самосинхронизирующаяся схема (вопрос 3, принцип 2) с d = 8.

Распространение ошибки. Пусть при передаче искажён один бит в сегменте y_j: получатель принял $y'_j = y_j \oplus e$, где e — 8-битовый вектор с одной единицей. Остальные сегменты приняты верно.

1. Сегмент j. Регистр R_j содержит сегменты $y_{j-8}, \ldots, y_{j-1}$ (при $j \le 8$ — ещё и часть IV), принятые без ошибок, поэтому гамма γ_j верная, и $x'_j = y'_j \oplus \gamma_j = x_j \oplus e$. В сегменте x_j искажён ровно один бит — в той же позиции, что и в шифртексте.
2. Сегменты с j + 1 по j + 8. Искажённый сегмент y'_j попадает в регистр на шаге j + 1 и остаётся в нём восемь шагов (таблица 5). На каждом из них

регистр получателя отличается от регистра отправителя, и блочный шифр выдаёт другой 64-битовый блок. У хорошего блочного шифра изменение входа меняет выход непредсказуемо, поэтому неверная гамма γ'_{j+t} не связана с верной γ_{j+t}, и сегмент $x'_{j+t} = y_{j+t} \oplus \gamma'_{j+t}$ искажается непредсказуемо: каждый его бит неверен с вероятностью 1/2, то есть в среднем неверны 4 бита из 8. Совпасть с верным сегмент может лишь случайно, с вероятностью 2^{-8}.

3. Сегменты начиная с j + 9. Регистр $R_{j+9} = y_{j+1} \| \dots \| y_{j+8}$ уже не содержит искажённого сегмента, получатель снова вырабатывает верную гамму, и дальше текст расшифровывается правильно: синхронизация восстановилась сама.

Таблица 5 – Положение искажённого сегмента в регистре при расшифровании (разряды регистра пронумерованы слева направо от 1 до 64)

Шаг	**Сегменты в регистре**	**Разряды с y'_j**	**Сегмент открытого текста**
j	$y_{j-8} \dots y_{j-1}$	—	x_j: искажён 1 бит
j + 1	$y_{j-7} \dots y_{j-1}, y'_j$	57–64	x_{j+1}: искажён непредсказуемо
j + 2	$y_{j-6} \dots y'_j, y_{j+1}$	49–56	x_{j+2}: искажён непредсказуемо
j + 3	$y_{j-5} \dots y'_j \dots y_{j+2}$	41–48	x_{j+3}: искажён непредсказуемо
j + 4	$y_{j-4} \dots y'_j \dots y_{j+3}$	33–40	x_{j+4}: искажён непредсказуемо
j + 5	$y_{j-3} \dots y'_j \dots y_{j+4}$	25–32	x_{j+5}: искажён непредсказуемо
j + 6	$y_{j-2} \dots y'_j \dots y_{j+5}$	17–24	x_{j+6}: искажён непредсказуемо
j + 7	$y_{j-1}, y'_j \dots y_{j+6}$	9–16	x_{j+7}: искажён непредсказуемо
j + 8	$y'_j, y_{j+1} \dots y_{j+7}$	1–8	x_{j+8}: искажён непредсказуемо
j + 9	$y_{j+1} \dots y_{j+8}$	вытеснен	x_{j+9}: верный

Подсчёт искажённых сегментов. Искажается 1 + 64/8 = 9 сегментов открытого текста по 8 бит, то есть затронуты 72 подряд идущих бита: в первом сегменте неверен ровно один бит, а следующие восемь расшифрованы с неверной гаммой и искажены непредсказуемо (в среднем половина битов). В общем виде, когда длина блока n кратна длине сегмента s, одна ошибка искажает 1 + n/s сегментов.

Пересчёт в 64-битовые блоки. Если открытый текст разбит на 64-битовые блоки, считая от начала сообщения, в каждом блоке 8 сегментов.

Пусть искажённый сегмент j стоит в своём 64-битовом блоке на позиции $p \in \{0, 1, \ldots, 7\}$. Девять искажённых сегментов занимают позиции p, p + 1, …, p + 8, считая от начала этого блока. Так как $8 \le p + 8 \le 15$, последний искажённый сегмент всегда попадает в следующий блок и никогда не выходит за его пределы. Значит, при любом положении ошибки искажаются ровно два соседних 64-битовых блока. Например, при p = 0 в первом блоке искажены один бит первого байта и следующие семь байтов, а во втором — только первый байт; при p = 7 в первом блоке искажён один бит последнего байта, а во втором — все восемь байтов.

Ответ. Искажаются 9 восьмибитовых блоков открытого текста (72 бита): в первом — один бит, в том же разряде, что и в шифртексте, восемь следующих искажаются непредсказуемо. Эти 72 бита всегда лежат в двух соседних 64-битовых блоках открытого текста, то есть искажаются 2 блока по 64 бита. Исключение — ошибка в одном из последних восьми сегментов сообщения: тогда искажённых сегментов меньше девяти, потому что сообщение заканчивается раньше, и может быть затронут только один 64-битовый блок.

Для сравнения: в CFB с полной обратной связью (сегмент 64 бита) одна ошибка искажает один бит текущего блока и непредсказуемо весь следующий блок, в режиме простой замены — непредсказуемо один блок, а в OFB — только один бит.

Вопрос 6. Длина значения хэш-функции и вероятность коллизии

Задание: однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^{-9}? Обязательно должен быть приведён ход решения.

Модель. Хэш-функция h отображает сообщения в множество V_n двоичных векторов длины n, в котором $N = 2^n$ элементов. Коллизия — пара

разных сообщений $x_1 \neq x_2$ с $h(x_1) = h(x_2)$. Используем вероятностную модель, в которой выводится парадокс дней рождений: значения хорошей хэш-функции на разных сообщениях ведут себя как независимые случайные величины, равномерно распределённые на V_n (модель случайной функции). Сообщения играют роль людей, значения хэш-функции — дней рождения, N — числа дней в году.

Вероятность коллизии среди m сообщений. Вероятность того, что значения хэш-функции на m сообщениях попарно различны, равна

$$P_{\text{нет}}(m) = \frac{N}{N} \cdot \frac{N-1}{N} \cdots \frac{N-m+1}{N} = \prod_{i=1}^{m-1} \left(1 - \frac{i}{N}\right),$$

поэтому вероятность хотя бы одной коллизии

$$P(m) = 1 - \prod_{i=1}^{m-1} \left(1 - \frac{i}{N}\right) \approx 1 - e^{-m(m-1)/(2N)}.$$

В этом и состоит парадокс дней рождений: P(m) становится заметной уже при m порядка √N.

Случайная пара сообщений. В задаче сообщений два, m = 2, и произведение состоит из одного множителя:

$$P(2) = 1 - \left(1 - \frac{1}{N}\right) = \frac{1}{N} = 2^{-n}.$$

Тот же результат получается напрямую: значение $h(x_1)$ какое-то, а $h(x_2)$ совпадёт с ним с вероятностью 1/N, так как все N значений равновероятны:

$$P\{h(x_1) = h(x_2)\} = \sum_{v \in V_n} P\{h(x_1) = v\} \cdot P\{h(x_2) = v\} = N \cdot \frac{1}{N} \cdot \frac{1}{N} = \frac{1}{N}.$$

Неравенство и его решение. По условию

$$2^{-n} \leq 10^{-9}.$$

1. Перейдём к обратным величинам: $2^n \geq 10^9$.
2. Прологарифмируем по основанию 2; логарифм — возрастающая функция, поэтому знак неравенства сохраняется: $n \geq \log_2 10^9 = 9 \cdot \log_2 10$.
3. Вычислим $\log_2 10 = \ln 10 / \ln 2 \approx 2{,}302585 / 0{,}693147 \approx 3{,}321928$.
4. Получаем $n \geq 9 \cdot 3{,}321928 \approx 29{,}897$.
5. Размерность n — целое число, поэтому $n \geq 30$.

Проверка.

- $n = 29$: $2^{29} = 536\,870\,912 < 10^9$, вероятность коллизии $2^{-29} \approx 1{,}86 \cdot 10^{-9} > 10^{-9}$ — условие не выполнено.
- $n = 30$: $2^{30} = 1\,073\,741\,824 \geq 10^9$, вероятность коллизии $2^{-30} \approx 9{,}31 \cdot 10^{-10} \leq 10^{-9}$ — условие выполнено.

Ответ: $n \geq 30$; наименьшая подходящая размерность — $n = 30$.

Замечание. Условие $n \geq 30$ гарантирует малую вероятность коллизии только для одной случайной пары. Противник, который ищет коллизию, перебирает много сообщений, а среди m сообщений уже $m(m-1)/2$ пар. По парадоксу дней рождений при $n = 30$ коллизия с вероятностью 1/2 находится примерно среди $m \approx \sqrt{(2N \ln 2)} \approx 1{,}18 \cdot 2^{15} \approx 38\,600$ сообщений. Если противник может вычислить h для m сообщений, вероятность коллизии не больше $m(m-1)/2 \cdot 2^{-n}$ (вероятность объединения событий не больше суммы их вероятностей), и условие принимает вид

$$\frac{m(m-1)}{2} \cdot 2^{-n} \leq 10^{-9}, \quad \text{то есть} \quad n \geq \log_2 \frac{m(m-1)}{2} + 9\log_2 10.$$

При $m = 2$ оно совпадает с решённым выше, а при $m = 2^{64}$ даёт $n \geq 127 + 29{,}9 \approx 157$. Атака дней рождений находит коллизию примерно за $2^{n/2}$ вычислений, поэтому у SHA-256 и «Стрибога» длина значения 256 или 512 бит.